

Eingereicht von
Tobias Rafetseder
12306137

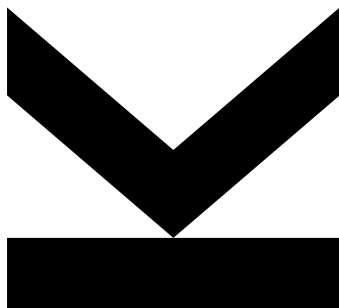
Angefertigt am
Institut für
Wirtschaftsinformatik -
Data and Knowledge
Engineering

Betreuer
o. Univ.-Prof. Dipl.-Ing. Dr.
Michael Schrefl

Mitbetreuung
Dipl.-Ing.
Sebastian Windsperger

Juni 2026

Ein ASP-basierter Ansatz zur Lösung des Nurse Scheduling Problems mit Clingo



Bachelorarbeit

zur Erlangung des akademischen Grades

Bachelor of Science

im Bachelorstudium

Wirtschaftsinformatik

Kurzfassung

Space for your abstract.

Kurzfassung

Platz für deine Kurzfassung.

Inhaltsverzeichnis

Kurzfassung	ii
Kurzfassung	iii
1 Einleitung	1
1.1 Einleitung	1
1.2 Ziel	1
2 Grundlagen	3
2.1 Nurse Scheduling Problem	3
2.2 Answer Set Programming	4
2.2.1 Syntax und Semantik	4
2.2.2 Answer Sets als Lösungskonzept	4
2.2.3 Verarbeitung und Werkzeuge	5
2.2.4 Anwendungsgebiete	5
2.3 Clingo	5
3 Vergleich bestehender Systeme	7
3.1 Das ASP-basierte Nurse Scheduling System an dem Krankenhaus der Universität von Yamanashi	7
3.2 ASP-basierte Nurse (Re)Scheduling-Ansätze in einem italieni- schen Krankenhaus	8
3.3 Ein ASP-basiertes Rehabilitationsplanungssystem für die ICS Maugeri	8
4 Anforderungen, Systemarchitektur und Datenmodell	10
4.1 Anforderungen	10
4.2 Systemarchitektur	11
4.3 Datenmodell	12
4.3.1 Entitäten	12
4.3.2 Assoziationstabellen	12
5 REST-Schnittstellen und DTOs	14
5.1 REST-Schnittstellen	14
5.2 Data Transfer Objects	15
6 ASP-Logik	17
6.1 Fakten-Generierung	17
6.2 Regelwerk	18
6.3 Zweiphasen-Lösungsanstz	18
6.4 Rescheduling-Logik	18
6.5 Timeout NP-hartes Problem	18
7 Conclusion and Outlook	19

Inhaltsverzeichnis	v
Literatur	20
Anhang A An Appendix	21

Kapitel 1

Einleitung

1.1 Einleitung

Die Erstellung von Dienstplänen in Krankenhäusern stellt eine der komplexesten organisatorischen Aufgaben im Gesundheitswesen dar. In den Abteilungen müssen zahlreiche Anforderungen gleichzeitig berücksichtigt werden: Qualifikationen und Verfügbarkeiten des Pflegepersonals, gesetzliche Arbeitszeitregelungen, sowie individuelle Präferenzen der Mitarbeitenden. Kurzfristige Personalausfälle erhöhen die Komplexität zusätzlich und machen eine rein manuelle Planung zunehmend ineffizient. Die händische Erstellung von Dienstplänen ist nicht nur zeitaufwendig, sondern auch fehleranfällig und bietet wenig Skalierbarkeit bei wachsenden Anforderungen. Die Informatik stellt seit Jahrzehnten formale Methoden zur Lösung derartiger kombinatorischer Optimierungsprobleme bereit. Das sogenannte Nurse Scheduling Problem (NSP) ist in der wissenschaftlichen Literatur gut etabliert und wurde mit einer Vielzahl von Ansätzen untersucht, zum Beispiel mit Constraint Programming. In der praktischen Anwendung mangelt es jedoch häufig an Lösungen, die gleichermaßen flexibel gegenüber abteilungsspezifischen Anforderungen, transparent in ihrer Modellierung und nachhaltig wartbar sind. Answer Set Programming (ASP) bietet in diesem Kontext einen vielversprechenden Ansatz. Als deklaratives Paradigma der Logikprogrammierung ermöglicht ASP eine präzise und gut lesbare Modellierung von Constraints und Optimierungszielen, ohne dass problemspezifische Suchalgorithmen explizit implementiert werden müssen. Mit Clingo, einem der leistungsfähigsten und in der Forschung weitverbreitetsten ASP-Solver, steht ein effizientes Werkzeug zur Verfügung.

1.2 Ziel

Ziel dieser Bachelorarbeit ist die Entwicklung eines ASP-basierten Ansatzes zur automatisierten Erstellung und kurzfristigen Anpassung von Dienstplänen in der Augenabteilung eines Krankenhauses, sowie dessen Umsetzung als vollständige Fullstack-Webanwendung. Die Applikation umfasst ein benutzerorientiertes Frontend zur Verwaltung von Personal, Stationen und Kompetenzen sowie ein Backend, das die Optimierungslogik mittels Clingo ausführt und die generierten Dienstpläne strukturiert zurückliefert. Sowohl harte Constraints, wie Mindestbesetzung und Qualifikationsanforderungen, als auch weiche Constraints, wie eine gleichmäßige Verteilung der Zuteilungen zu Stationen, werden dabei formal modelliert und automatisiert gelöst. Die Arbeit un-

tersucht damit, inwieweit ASP als Grundlage für eine in der Praxis verwendbare, vollständig integrierte Planungssoftware im Gesundheitssektor eingesetzt werden kann.

Kapitel 2

Grundlagen

2.1 Nurse Scheduling Problem

Das *Nurse Scheduling Problem* (NSP) beschreibt die Zuordnung von Arbeitsschichten für Pflegekräfte innerhalb eines bestimmten Planungszeitraums. Ziel ist es, die Abfolge der Schichten für jede Pflegekraft so festzulegen, dass eine kontinuierliche und angemessene Patientenversorgung gewährleistet wird. Die Erstellung solcher Dienstpläne stellt eine komplexe und zeitaufwendige Aufgabe dar. Gleichzeitig beeinflussen Dienstpläne mit unterschiedlichen Schichten das Stressniveau der Pflegekräfte sowie deren soziale Beziehungen und Familienleben erheblich (Oughalime2008).

Das Nurse Scheduling Problem umfasst die Zuweisung Schichten unter Berücksichtigung verschiedener Nebenbedingungen. Dabei muss sichergestellt werden, dass jederzeit ausreichend Personal vorhanden ist, ohne unnötige Überbesetzung zu verursachen. Die Dienstplanung erfordert daher umfangreiches Wissen und Erfahrung und wird in vielen Krankenhäusern manuell durch Stationsleitungen oder andere verantwortliche Personen durchgeführt. Ein geeigneter Dienstplan soll nicht nur die täglichen Schichtanforderungen erfüllen, sondern auch Fairness zwischen den Pflegekräften gewährleisten und ihre Gesundheit sowie ihr Privatleben möglichst wenig beeinträchtigen (Oughalime2008).

Zur Lösung des Nurse Scheduling Problems wurden verschiedene mathematische und heuristische Verfahren entwickelt. In der Literatur werden unter anderem Goal Programming, Simulated Annealing und Tabu Search eingesetzt. Ziel dieser Verfahren ist es, sowohl die Anforderungen des Krankenhauses als auch die Präferenzen der Pflegekräfte zu berücksichtigen (Oughalime2008). Der in der vorliegenden Arbeit betrachtete Ansatz basiert auf einer ASP-basierten Lösung.

Hard Constraints müssen in jedem Fall erfüllt werden, da der Dienstplan ansonsten als unzulässig gilt. Dazu gehört beispielsweise die Sicherstellung einer Mindestanzahl an Pflegekräften pro Schicht und Tag. Außerdem darf eine Pflegekraft pro Tag nur einer Schicht zugewiesen werden. *Soft Constraints* repräsentieren die Wünsche und Präferenzen der Pflegekräfte. Diese sollen möglichst erfüllt werden, sind jedoch nicht zwingend erforderlich. Dazu gehört beispielsweise die bevorzugte Anzahl an Arbeitstagen pro Planungszeitraum. Außerdem sollen ungünstige Schichtfolgen vermieden werden (Oughalime2008).

2.2 Answer Set Programming

Answer Set Programming (ASP) ist ein deklarativer Ansatz zur Modellierung und Lösung von Such- und Optimierungsproblemen. Es entstand aus der Schnittmenge mehrerer Forschungsgebiete, insbesondere der Wissensrepräsentation, der Logikprogrammierung und der Constraint-Satisfaction, und vereint deren Stärken in einem einheitlichen Framework. ASP erlaubt es, Probleme deklarativ zu formulieren, ohne dass der Programmierer den Suchprozess selbst steuert.

2.2.1 Syntax und Semantik

Die grundlegenden Bausteine eines ASP-Programms sind Atome, Literale und Regeln. Atome sind elementare Aussagen mit einem Wahrheitswert, Literale sind entweder Atome oder deren Negation. Regeln haben die allgemeine Form:

```
1 a ← b1, . . . , bm, not c1, . . . , not cn
```

Dabei bezeichnet a , also die linke Seite der Regel den Regelkopf, und die rechte Seite den Regelrumpf. Die Regel ist so zu interpretieren: Atom a gilt, sofern alle b_m ableitbar sind und keines der c_n ableitbar ist. Der Operator `not` ist nicht als klassische Negation zu verstehen, sondern als Default-Negation, die die Nichtableitbarkeit eines Atoms ausdrückt. Als Beispiel:

```
1 broken ← lightning, not lightning_rod
2 light_on ← power_on, not broken
```

Hier leuchtet die Lampe, wenn der Strom an ist und es keinen Grund gibt anzunehmen, dass sie kaputt ist. Ist nichts über einen Defekt bekannt, gilt `not broken`, und die Lampe leuchtet. Wenn `lightning` als Fakt hinzukommt, wird `broken` ableitbar, und `not broken` gilt nicht mehr, also leuchtet die Lampe nicht mehr. Da ASP eng mit der nichtmonotonen Logik verwandt ist, können bereits gezogene Schlussfolgerungen zurückgezogen werden, sobald neue Fakten oder Regeln hinzugefügt werden. Die Default-Negation ist dabei der zentrale Mechanismus, der dieses Verhalten ermöglicht.

2.2.2 Answer Sets als Lösungskonzept

Die Semantik von ASP-Programmen basiert auf dem Begriff des Answer Sets, auch früher unter dem Namen *stable models* bekannt, der auf **stableModelSemanticGelfond** zurückgeht. Ein Answer Set ist eine Menge von Atomen, die in sich konsistent ist und bei der jedes enthaltene Atom durch eine Regelanwendung begründet werden kann. Ein Programm kann dabei mehrere, genau eines oder kein Answer Set besitzen, wobei jedes Answer Set einer gültigen Lösung des modellierten Problems entspricht (Brewka et al., 2011). Diese Mehrwertigkeit macht ASP besonders geeignet für Probleme, bei denen mehrere gleichwertige Lösungen existieren können, wie zum Beispiel bei der Dienstplanung.

2.2.3 Verarbeitung und Werkzeuge

Die Ausführung eines ASP-Programms erfolgt typischerweise in zwei Schritten. Zunächst übersetzt ein sogenannter *Grounder* das Programm bestehend aus Prädikaten in eine äquivalente aussagenlogische Form, indem alle Variablen durch konkrete Konstanten ersetzt werden. Anschließend berechnet ein *Solver* die Answer Sets des resultierenden Programms. Bekannte Systeme sind unter anderem CLASP, DLV und SMODELs. Moderne Solver nutzen dabei Techniken aus dem SAT-Solving, die sicherstellt, dass jedes wahre Atom tatsächlich ableitbar ist (Brewka et al., 2011).

2.2.4 Anwendungsgebiete

Aufgrund seiner Ausdruckstärke und Flexibilität findet ASP Anwendung in einer Vielzahl von Anwendungsbereichen. Industriell wurde es beispielsweise zur automatisierten Schichtplanung im Containerhafen Gioia Tauro eingesetzt, wo ein ASP-basiertes System die manuelle Planung von mehreren Stunden auf wenige Minuten reduzierte und gleichzeitig die Planqualität verbesserte. Weitere Anwendungsfelder umfassen Entscheidungsunterstützungssysteme, die Analyse biologischer Netzwerke, phylogenetische Systematik sowie klassische kombinatorische Probleme wie den Hamiltonkreis (Brewka et al., 2011).

2.3 Clingo

Clingo ist ein modernes ASP-System zur Berechnung von *stable models* und erweitert klassische ASP-Systeme um das Konzept des Multi-Shot Solving. Während frühere Systeme einem einmaligen („one-shot“) Ablauf folgen, der aus Grounding und anschließendem Solving besteht, ermöglicht Clingo die Verarbeitung sich kontinuierlich verändernder Logikprogramme innerhalb eines laufenden Prozesses. Dadurch können Programme schrittweise erweitert, angepasst oder deaktiviert werden, ohne den Grounder oder Solver neu starten zu müssen (Gebser et al., 2018).

Ein zentrales Merkmal von Clingo ist die Integration von Grounder und Solver innerhalb eines einzigen Systems. Frühere Versionen von Clingo bestanden aus der Kombination des Grounders gringo und des Solvers clasp. Mit der Einführung des Multi-Shot Solving wurde Clingo jedoch um Mechanismen erweitert, die einen flexiblen und zustandsbasierten Lösungsprozess erlauben. Dabei bleibt der interne Zustand des Systems erhalten und kann durch verschiedene Operationen verändert werden. Dies reduziert Redundanzen beim erneuten Grounding und ermöglicht gleichzeitig die Wiederverwendung bereits gelernter Informationen des Solvers (Gebser et al., 2018).

Ein weiterer wichtiger Bestandteil von Clingo ist die bereitgestellte Programmierschnittstelle (API). Diese erlaubt die Kontrolle des Grounding- und Solving-Prozesses aus einer prozeduralen Programmiersprache heraus. Clingo stellt hierfür eine C-API bereit, sowie Bindings für Python und Lua (Gebser et al., 2018). Besonders relevant für diese Arbeit ist die Integration von Python,

da sie eine flexible Steuerung komplexer Reasoning-Prozesse innerhalb von Clingo ermöglicht.

Die Integration von beispielsweise Python erfolgt über das *Directive* `#script(python)`. Innerhalb eines solchen Skriptblocks kann eine *main*-Funktion definiert werden, die Zugriff auf ein Kontrollobjekt erhält. Über dieses Objekt können Subprogramme gezielt *gegrounded* und *Solving*-Aufrufe gestartet werden. Beispielsweise kann mit `prg.ground(...)` ein bestimmtes Teilprogramm instanziiert und mit `prg.solve()` die Berechnung von *Answer Sets* ausgelöst werden. Dadurch wird die deklarative Wissensrepräsentation von ASP mit prozeduraler Steuerungslogik kombiniert (Gebser et al., 2018).

Kapitel 3

Vergleich bestehender Systeme

Im Zuge der Entwurfsphase des NSP-Solvers wurde eine Literaturrecherche zu bestehenden Systemen und Ansätzen durchgeführt. Auf ähnliche ASP-basierte Implementierungen im Gesundheitssektor wird in den nächsten Punkten eingegangen.

3.1 Das ASP-basierte Nurse Scheduling System an dem Krankenhaus der Universität von Yamanashi

An dem Krankenhaus der Universität von Yamanashi wurde ein *Nurse Scheduling System* mit Clingo umgesetzt. Dabei wurden Einschränkungen wie *Workdays*, *Weekly Rest Days*, *Requested Shifts*, *Average Working Hours*, *Consecutive Workdays*, *Shift Frequency* sowie *Daily Staffing* berücksichtigt. Das dort entwickelte System wird als Web-Service bereitgestellt, was Pflegeleitungen erlaubt, über den Browser Dienstpläne zu erstellen und zu modifizieren. Fokus der Arbeit war das Erstellen von komplexen Dienstplänen alle vier Wochen (Nabeshima et al., 2026).

Die Applikation unterstützt nicht nur die automatische Erstellung von Dienstplänen, sondern auch eine interaktive Nachbearbeitung und Anpassung bestehender Pläne. Pflegeleitungen können einzelne Schichten manuell ändern, gewünschte oder unerwünschte Dienste definieren sowie bestimmte Zuweisungen fixieren, damit diese bei späteren Optimierungsschritten erhalten bleiben. Dies ist besonders relevant, da in der Praxis häufig kurzfristige Änderungen durch Krankheitsausfälle oder andere unvorhersehbare Ereignisse auftreten und somit ein *Rescheduling* erforderlich wird (Nabeshima et al., 2026).

Ein weiterer Kernaspekt des Systems ist die Minimierung unnötiger Änderungen an bereits bestehenden Dienstplänen. Dadurch soll der Aufwand für die Überprüfung der Pläne reduziert und vermieden werden, dass Pflegekräfte ständig über größere Änderungen informiert werden müssen. Zur Umsetzung dieses Ansatzes wird die Methode *Large Neighborhood Prioritized Search (LNPS)* eingesetzt. Im Gegensatz zu klassischen Verfahren der Minimal Perturbation werden dabei bestehende Schichtzuweisungen bevorzugt beibehalten, ohne sie jedoch vollständig zu fixieren. Dadurch bleibt die Lösung flexibel genug, um weiterhin Optimierungen durchführen und neue Randbedingungen berücksichtigen zu können. Die Priorisierung wird mithilfe von Heuristiken des ASP-Solvers Clingo umgesetzt (Nabeshima et al., 2026).

3.2 ASP-basierte Nurse (Re)Scheduling-Ansätze in einem italienischen Krankenhaus

Für ein italienisches Krankenhaus wurde ein ASP-basierter Ansatz zur Lösung des *Nurse Scheduling Problems* entwickelt. Dabei werden Dienstpläne für ein gesamtes Jahr mit drei verschiedenen Schichttypen (Früh-, Spät- und Nachtschicht) erstellt. Berücksichtigt werden unter anderem Einschränkungen bezüglich der maximalen Arbeitsstunden pro Jahr sowie Vorgaben zur Häufigkeit bestimmter Schichtarten. Ein besonderer Fokus der Arbeit lag auf der Optimierung der ASP-Kodierung, da frühere Ansätze durch große Aggregate die Effizienz moderner ASP-Solver negativ beeinflussten. Die neue Kodierung berücksichtigt deshalb nur Kombinationen von Werten, die zu gültigen Dienstplänen führen können, wodurch die Suchraumreduktion verbessert wird (Alviano et al., 2019).

Zur Evaluierung wurden die ASP-Solver *clingo* und *wasp* eingesetzt und zusätzlich mit SAT-, MaxSAT- sowie ILP-basierten Ansätzen verglichen. Die Ergebnisse zeigen, dass insbesondere *clingo* mit der erweiterten ASP-Kodierung die besten Laufzeiten erzielt. Während frühere Kodierungen mehrere Minuten bis Stunden benötigten, konnten mit der verbesserten Variante Dienstpläne deutlich schneller berechnet werden. Auch die Skalierbarkeit wurde untersucht, indem Instanzen mit bis zu 164 Pflegekräften betrachtet wurden (Alviano et al., 2019).

Neben der eigentlichen Dienstplanung wurde auch das *Nurse Rescheduling* untersucht, das kurzfristige Ausfälle von Pflegekräften berücksichtigt. Ziel des Ansatzes ist es, bestehende Dienstpläne möglichst wenig zu verändern, um organisatorischen Aufwand und Unzufriedenheit bei den Mitarbeitenden zu minimieren. Dafür werden neue Dienstpläne berechnet, die sowohl die ursprünglichen Anforderungen als auch die kurzfristigen Abwesenheiten berücksichtigen. Die Experimente zeigen, dass die ASP-basierten Ansätze auch für solche Rescheduling-Szenarien einsetzbare Lösungen innerhalb weniger Sekunden bis Minuten berechnen können (Alviano et al., 2019).

3.3 Ein ASP-basiertes Rehabilitationsplanungssystem für die ICS Maugeri

Für die Rehabilitationskliniken der ICS Maugeri in Italien wurde ein ASP-basiertes System zur automatischen Erstellung von Rehabilitationsplänen entwickelt. Das System adressiert das *Rehabilitation Scheduling Problem* und plant physiotherapeutische Sitzungen für Patienten sowie die Zuweisung geeigneter Therapeuten. Dabei werden unterschiedliche Anforderungen berücksichtigt, zum Beispiel die Qualifikationen der Therapeuten, gesetzliche Arbeitszeiten, Mindestbehandlungszeiten, Raumkapazitäten, Patientenpräferenzen sowie eine faire Verteilung der Arbeitslast (*cardellini2023solvingrehabilitationschedulingproblems*).

Das System ist in zwei aufeinanderfolgende Phasen unterteilt. In der ersten Phase, dem sogenannten *Board*, werden Patienten passenden Therapeuten zugewiesen, wobei insbesondere Arbeitszeiten und Mindestbehandlungs-

dauern berücksichtigt werden. In der zweiten Phase, dem *Agenda*-Schritt, werden anschließend konkrete Termine, Start- und Endzeiten sowie verfügbare Räume für die Therapiesitzungen geplant. Zusätzlich werden Präferenzen der Patienten hinsichtlich bevorzugter Tageszeiten einbezogen. Beide Phasen wurden mithilfe separater ASP-Encodings mit dem Solver *Clingo* umgesetzt ([cardellini2023solvingrehabilitationschedulingproblems](#)).

Ein zentraler Aspekt des Systems ist die Möglichkeit zur manuellen Nachbearbeitung der Planung. Zwischen den beiden Phasen können Koordinatoren die im Board-Schritt erzeugten Zuweisungen überprüfen und bei Bedarf anpassen, bevor die eigentliche Terminplanung durchgeführt wird. Dieses Vorgehen orientiert sich am bisherigen manuellen Prozess der Kliniken und stellt sicher, dass medizinisches Personal weiterhin Kontrolle über die endgültige Planung behält ([cardellini2023solvingrehabilitationschedulingproblems](#)).

Das System wurde mit realen Daten aus den Kliniken Genova Nervi und Castel Goffredo evaluiert sowie zusätzlich anhand Benchmarks auf Skalierbarkeit untersucht. Zur Verbesserung der Performance wurden domänenspezifische Optimierungen eingesetzt, die insbesondere das Grounding reduzieren. Für die meisten Instanzen konnten optimale Lösungen innerhalb weniger Sekunden berechnet werden ([cardellini2023solvingrehabilitationschedulingproblems](#)).

Kapitel 4

Anforderungen, Systemarchitektur und Datenmodell

4.1 Anforderungen

Basierend auf bereits bestehenden Implementierungen sowie konkreten Anforderungen des Krankenhauses wurden die Systemanforderungen definiert. Das Projekt wird als Full-Stack-Webanwendung umgesetzt und ist über den Webbrowser zugänglich. Dabei kommen folgende Technologien zum Einsatz:

Tabelle 1: Technologien

Subsystem	Konkrete Technologie
Frontend	Angular
Backend	FastAPI
Datenbank	PostgreSQL / Supabase
ASP-Solver	Clingo

Das System soll die Erstellung eines Monatsplans von der Pflegeleitung für Mitarbeiter in einer Abteilung in einem Krankenhaus ermöglichen. Für jeden Mitarbeiter soll die Anzahl der Arbeitstage pro Monat erfasst werden können. Im Rahmen dieser Arbeit wird ausschließlich eine Tagesschicht betrachtet, unterschiedliche Schichtarten werden daher nicht berücksichtigt.

Zusätzlich müssen verschiedene Hard Constraints eingehalten werden. Ein Mitarbeiter darf an einem Tag nur einer Station zugeteilt werden. Außerdem darf eine Zuteilung nur erfolgen, wenn der Mitarbeiter über die erforderlichen Kompetenzen für die jeweilige Station verfügt. Bestimmte Zuteilungen von Mitarbeitern zu Tagen und Stationen sollen als fixiert markiert werden können, sodass diese bei einer Neuberechnung des Plans unverändert bleiben. Ebenso soll definiert werden können, dass ein Mitarbeiter an bestimmten Tagen nicht eingeteilt werden darf, oder an bestimmten Tagen eingeteilt werden muss.

Neben den verpflichtenden Einschränkungen sollen auch Soft Constraints berücksichtigt werden. Als Soft Constraint wurde eine möglichst faire Verteilung der Mitarbeiter auf die einzelnen Stationen definiert. Wenn eine gültige Lösung existiert, bei der jede Station an jedem Tag mit mindestens einem Mitarbeiter

besetzt ist, soll das System eine möglichst gleichmäßige Zuteilung der Mitarbeiter auf die Stationen anstreben.

Ein weiterer wichtiger Punkt ist die Berechnungszeit des Systems. Vergleichbare Systeme berechnen Monatspläne innerhalb einer Minute, während Jahrespläne mehrere Minuten benötigen. Das entwickelte System soll daher ebenfalls in der Lage sein, einen Monatsplan innerhalb weniger Minute zu berechnen. Auch ein krankheitsbedingtes *Rescheduling* soll innerhalb weniger Minuten möglich sein. Falls keine gültige Lösung gefunden werden kann, soll das System den Benutzer entsprechend warnen.

4.2 Systemarchitektur

Die Applikation besteht aus einer dreischichtigen Architektur, die in Angular-Frontend, FastAPI-Backend sowie einer Supabase-Datenbank mit PostgreSQL aufgeteilt wird.

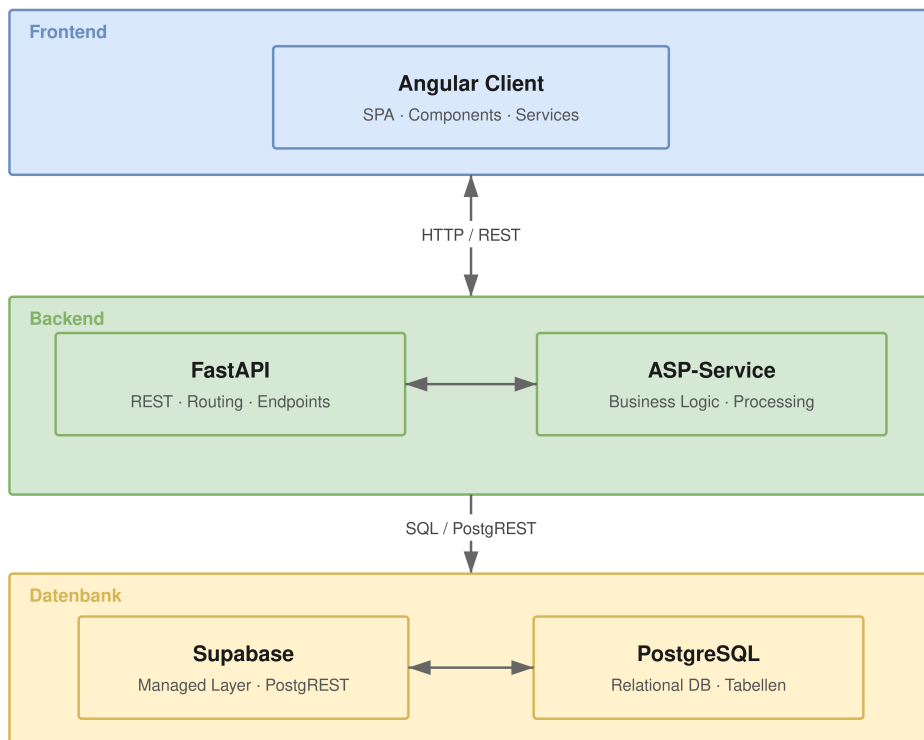


Abbildung 1: Applikations-Architektur

Das Frontend wurde als Single-Page Application mit dem Angular Framework umgesetzt. Es stellt die Benutzeroberfläche bereit, über die Mitarbeiter, Stationen, Kompetenzen verwaltet, sowie Constraints festgelegt und Monatspläne angezeigt werden können. Die Kommunikation mit dem Backend erfolgt ausschließlich über HTTP-REST-Schnittstellen. Das Backend ist in Python mit FastAPI realisiert und übernimmt die gesamte Geschäftslogik der An-

wendung. Es stellt REST-Endpunkte bereit und verarbeitet eingehende Anfragen. Ein zentraler Bestandteil des Backends ist der ASP-Service. Dieser Service ist für die Berechnung der Monatspläne unter Berücksichtigung der definierten Hard und Soft Constraints verantwortlich. Als Datenbankschicht wird Supabase eingesetzt, eine Open-Source-Plattform, die auf PostgreSQL aufbaut. Supabase stellt neben der relationalen Datenbank auch eine PostgREST-Schnittstelle bereit, über die das Backend direkt auf die Datenbankressourcen zugreifen kann.

4.3 Datenmodell

4.3.1 Entitäten

In Abbildung 2 wird das Datenmodell dargestellt. Im Folgenden wird auf die Struktur der Daten eingegangen.

`users` repräsentiert die Mitarbeitenden des Systems. Neben einem Primärschlüssel (`id`) wird auch der Name (`name`) persistiert. Die Entität dient als Ankerpunkt für Qualifikations- und Zuweisungsbeziehungen.

`skills` bildet die Menge verfügbarer Kompetenzen ab. Jede Kompetenz beinhaltet eine `id` und einen `name`.

`stations` modelliert die Arbeitsstationen, denen Benutzer zugewiesen werden können. Das Attribut `maxAssignments` legt die maximale Anzahl gleichzeitiger Zuweisungen pro Station fest und stellt damit eine wichtige Kapazitätsbeschränkung dar.

Durch die Tabelle `schedules` ist es möglich, `Assignments` zu Diensplänen zuzuweisen und diese anhand der Attribute `year` und `month` zeitlich einzuordnen. Mit dem Boolean `is_loaded` wird gespeichert, welcher Dienstplan aktuell im Kalender geladen ist.

4.3.2 Assoziationstabellen

`user_skills` bildet die `m:n`-Beziehung zwischen `users` und `skills` ab. Dadurch kann gespeichert werden, über welche Qualifikationen ein Benutzer verfügt. Die Tabelle besteht ausschließlich aus den beiden Fremdschlüsseln `user_id` und `skill_id`.

Die Tabelle `station_skills` definiert, welche Fähigkeiten für eine bestimmte Station erforderlich sind, und bildet somit eine `m:n`-Beziehung zwischen `stations` und `skills`. Eine Station kann dabei mehrere benötigte Qualifikationen besitzen. Dadurch wird ein Abgleich zwischen den Fähigkeiten der Benutzer und den Anforderungen der Stationen ermöglicht.

`station_assignments` stellt die zentrale Tabelle des Datenmodells dar. Sie verbindet eine Station (`station_id`) mit einem Benutzer (`user_id`) sowie einem Dienstplan (`schedule_id`) für ein bestimmtes Datum (`date`). Das Attribut `user_id` ist optional, wodurch auch Platzhalter-Zuweisungen abgespeichert werden können, falls ein Dienstplan nicht für jede Station einen Mitarbeiter zuweisen kann.



Abbildung 2: Datenmodell

Kapitel 5

REST-Schnittstellen und DTOs

Dieses Kapitel beschreibt die REST-Schnittstellen sowie die verwendeten Data Transfer Objects (DTOs) der Anwendung, die für das *Scheduling* nötig sind. Die Schnittstellen bilden die Grundlage für die Kommunikation zwischen Frontend und Backend und ermöglichen den Austausch von Daten für die Dienstplanung sowie die Verwaltung von Benutzern, Stationen und Kompetenzen.

5.1 REST-Schnittstellen

Die nachfolgende Übersicht zeigt die implementierten REST-Endpunkte des Systems. Die Schnittstellen orientieren sich an den Prinzipien von REST und stellen Funktionen für das Erstellen, Laden, Aktualisieren und Löschen der wichtigsten Entitäten bereit.

Tabelle 2: REST-Schnittstellen

Method	Path	Description
<i>Schedule</i>		
POST	/api/v1/schedule	Dienstplan neu erstellen
POST	/api/v1/schedule/reschedule	Dienstplan neu berechnen
<i>Skills</i>		
GET	/api/v1/skills	Alle Skills laden
POST	/api/v1/skills	Neuen Skill anlegen
DELETE	/api/v1/skills/:id	Skill löschen
<i>Station Assignments</i>		
GET	/api/v1/station-assignments/all	Alle Zuweisungen laden
GET	/api/v1/station-assignments/:date	Zuweisungen nach Datum filtern
PUT	/api/v1/station-assignments/replace/:date	Zuweisungen eines Datums ersetzen
<i>Stations</i>		
GET	/api/v1/stations	Alle Stationen laden
GET	/api/v1/stations/:id	Station nach ID laden
POST	/api/v1/stations	Neue Station anlegen
PUT	/api/v1/stations/:id	Station aktualisieren
DELETE	/api/v1/stations/:id	Station löschen
<i>Users</i>		
GET	/api/v1/users	Alle Benutzer laden
GET	/api/v1/users/:id	Benutzer nach ID laden
POST	/api/v1/users	Neuen Benutzer anlegen
PUT	/api/v1/users/:id	Benutzer aktualisieren
DELETE	/api/v1/users/:id	Benutzer löschen

5.2 Data Transfer Objects

Für den Datenaustausch zwischen Frontend und Backend werden klar definierte Data Transfer Objects (DTOs) verwendet. Im Folgenden werden die wichtigsten Datenstrukturen für das *Scheduling* erläutert.

Der *SchedulePayload* definiert die Eingabedaten für die Dienstplanerstellung und wird bei jedem *Scheduling* und *Rescheduling* an das Backend übermittelt.

Tabelle 3: Felder von SchedulePayload

Feld	Typ	Beschreibung
currentMonth	number	Zielmonat der Planung (1–12)
currentYear	number	Zieljahr der Planung
daysInMonth	number	Anzahl der Tage im Zielmonat
keepExistingAssignments	boolean	Vorhandene Zuweisungen beibehalten
newPlan	boolean	Neuen Plan erstellen (default ist überschreiben)
alternativePlan	boolean	Alternativpläne berechnen
constraints	UserConstraint[]	Planungsrestriktionen der Benutzer

UserConstraint bildet die individuellen Planungsrestriktionen eines Benutzers ab und wird als Liste innerhalb des SchedulePayload mitgeschickt.

Tabelle 4: Felder von UserConstraint

Feld	Typ	Beschreibung
userId	string	Referenz auf den betreffenden Benutzer
maxDaysPerMonth	number	Maximale Einsatztage im Monat
minDaysPerMonth	number	Minimale Einsatztage im Monat
exactDaysPerMonth	number	Exakte Anzahl an Einsatztagen im Monat
fixedDays	number[]	Tage, an denen der Benutzer fix eingeteilt wird
blockedDays	number[]?	Tage, an denen der Benutzer nicht verfügbar ist

Kapitel 6

ASP-Logik

In diesem Kapitel wird genauer auf die ASP-Logik und die Generierung der Dienstpläne eingegangen.

6.1 Fakten-Generierung

Die Fakten-Generierung stellt die Verbindung zwischen der relationalen Datenbank und der ASP-Wissensbasis dar. Damit Clingo einen Dienstplan berechnen kann, müssen die benötigten Informationen zunächst in Form von ASP-Fakten bereitgestellt werden. Dafür werden die aus der Datenbank geladenen ORM-Objekte in der Funktion `_build_base_facts` in entsprechende Fakten umgewandelt.

Für jeden Benutzer wird dabei ein `user`-Fakt erzeugt. Zusätzlich wird für jede vorhandene Kompetenz ein `has_skill`-Fakt erstellt. Für Stationen werden ebenfalls Fakten generiert, darunter `station`, `max_assignments` zur maximalen Anzahl an Zuweisungen sowie `requires` für benötigte Kompetenzen.

Die zu planenden Arbeitstage werden als `day`-Fakten an Clingo übergeben. Wochenenden werden dabei bereits vorher durch die Funktion `get_weekdays` herausgefiltert. Zusätzlich werden benutzerspezifische Einschränkungen als Fakten festgelegt. Dazu zählen beispielsweise maximale oder minimale Arbeitstage pro Monat (`max_days`, `min_days`), fixe Arbeitstage (`fixed_day`) oder gesperrte Tage (`blocked_day`).

Im Folgenden wird beispielhaft ein Ausschnitt der generierten Basisfakten für ein *Scheduling* gezeigt.

```
1    user(1597).
2    has_skill(1597, 23).
3    has_skill(1597, 24).
4
5    user(1598).
6    has_skill(1598, 22).
7    has_skill(1598, 25).
8
9    station(865).
10   max_assignments(865, 3).
11   requires(865, 22).
12   requires(865, 23).
13
14   station(866).
```

```
15     max_assignments(866, 3).
16     requires(866, 24).
17
18     day(1).
19     day(4).
20     day(5).
21
22     exact_days(1597, 21).
23     exact_days(1598, 21).
```

Listing 6.1: Beispiel generierter Basisfakten

In diesem Beispiel werden zwei Benutzer, deren Kompetenzen, Stationen mit den benötigten Kompetenzen sowie Arbeitstage und Constraints als Fakten definiert. Auf Basis dieser Informationen kann Clingo anschließend gültige Dienstpläne berechnen.

6.2 Regelwerk

Das Regelwerk für die initiale Planerstellung ist in der Datei `rules.lp` definiert. Die Grundlage bildet die Bestimmung der *Eligibility*: Ein Benutzer gilt als qualifiziert für eine Station (`eligible/2`), wenn er alle erforderlichen Fähigkeiten besitzt, also kein `requires/2`-Fakt existiert, dem kein entsprechendes `has_skill/2`-Fakt gegenübersteht. Daraus wird `eligible_day/3` abgeleitet, welches zusätzlich gesperrte Tage ausschließt. Der *Entscheidungsraum* wird durch eine *Choice Rule* definiert, die Clingo erlaubt, Benutzern Stationen an bestimmten Tagen zuzuweisen, wobei pro Station und Tag maximal `Max` Benutzer zugewiesen werden dürfen. Die *harten Constraints* erzwingen, dass kein Benutzer an zwei Stationen gleichzeitig eingeteilt wird, dass gesperrte Tage nicht belegt werden, und dass benutzerspezifische Einschränkungen wie `max_days/2`, `min_days/2` sowie `exact_days/2` eingehalten werden. Fixe Arbeitstage werden über `fixed_day/2` als harte Bedingung erzwungen, sodass bestimmte Benutzer an vordefinierten Tagen zwingend eingeteilt sind. Die *Optimierung* minimiert unbesetzte Stationen über eine `#minimize`-Direktive, die Clingo anweist, die Anzahl der Tage, an denen eine Station ohne Besetzung bleibt, so gering wie möglich zu halten. Die Regelstruktur für das Rescheduling folgt einem ähnlichen Prinzip, wird jedoch in Abschnitt ?? gesondert behandelt.

6.3 Zweiphasen-Lösungsansatz

6.4 Rescheduling-Logik

6.5 Timeout NP-hartes Problem

Kapitel 7

Conclusion and Outlook

Space for your summary, central conclusions, and an outlook on potential future work.

Literatur

- Alviano, M., Dodaro, C., & Maratea, M. (2019). Nurse (Re)scheduling via answer set programming¹. *Intelligenza Artificiale*, 12(2), 109–124. <https://doi.org/10.3233/IA-170030>
- Brewka, G., Eiter, T., & Truszczyński, M. (2011). Answer set programming at a glance. *Communications of the ACM*, 54(12), 92–103. <https://doi.org/10.1145/2043174.2043195>
- Gebser, M., Kaminski, R., Kaufmann, B., & Schaub, T. (2018, 20. März). *Multi-shot ASP solving with clingo*. arXiv: 1705.09811 [cs]. <https://doi.org/10.48550/arXiv.1705.09811>
- Nabeshima, H., Banbara, M., Schaub, T., & Soh, T. (2026). The ASP-based Nurse Scheduling System at the University of Yamanashi Hospital. *Electronic Proceedings in Theoretical Computer Science*, 439, 334–348. <https://doi.org/10.4204/EPTCS.439.23>

Anhang A

An Appendix

Space for an appendix. You can have more than one appendix chapter. Appendices are, of course, optional.